Drug Tracking System

Data Engineering Pipeline — Technical Summary

DEPI Graduation Project
Team: TheSilence_DEPI

1. Project Overview

The Drug Tracking System helps patients find available drugs and nearby pharmacies with up-to-date pricing and availability information. Pharmacies continuously report stock levels and prices; the system aggregates this data so patients can compare availability and price across pharmacies, and find alternatives when a drug is scarce.

The project is divided by role across the following functions:

- Web Developer — patient-facing and pharmacy-facing interfaces
- Data Engineer — designs and builds the data pipeline (this document)
- Web Scrapers — collect supplementary drug and pricing data
- Data Wrangler — prepares and cleans raw data
- Dashboard / Analytics Table Designers — analytics views and dashboards

This document covers the Data Engineering component: ingesting pharmacy-submitted data, transforming it, and delivering clean, query-ready tables to the production database. The pipeline owns exactly seven tables: drug_categories, drugs, drug_alternatives, pharmacy_inventory, drug_analytics, price_history, and availability_history.

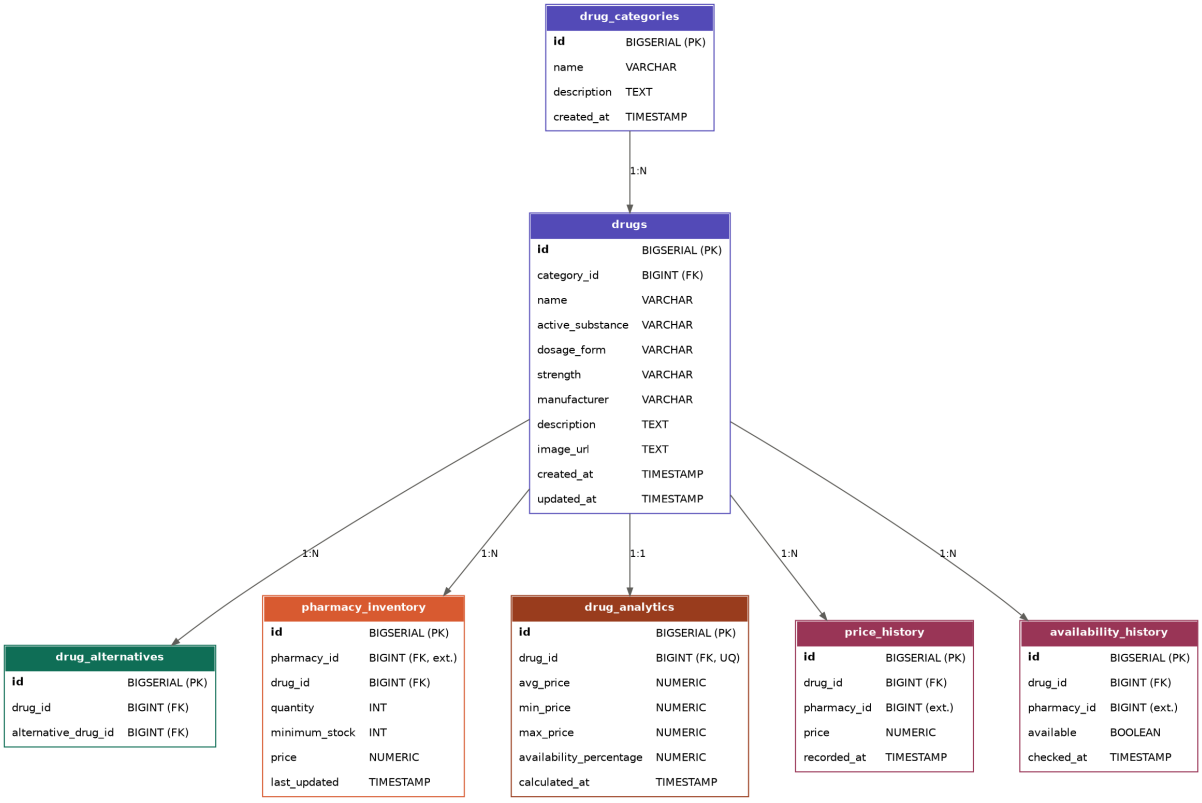
2. Architecture & Data Flow

Pharmacy files arrive in Supabase Storage, are pulled into a local working directory and a permanent dated archive, then pass through column normalization before being seeded into DuckDB. dbt transforms the data through staging, core, and analytics layers, and captures historical snapshots of price and availability changes. The final tables are pushed to a PostgreSQL production warehouse consumed by the backend API.

DuckDB serves as a fast, local, file-based transformation engine so that complex SQL transformations never run directly against the production database — only validated, final tables are pushed to PostgreSQL.

3. Database Schema (Entity-Relationship Diagram)

The diagram below shows the seven tables owned and maintained by the data engineering pipeline, along with their key columns, primary keys, and relationships. Fields marked (ext.) reference tables owned by other teams (e.g. pharmacies).



Key design notes: pharmacy_inventory has no stored availability column — availability is derived everywhere as quantity > 0. price_history and availability_history are populated automatically by dbt snapshots and are append-only.

4. Pipeline Execution

The pipeline runs as a six-task Airflow DAG, scheduled every 12 hours (midnight and noon), with two retries and a one-minute delay between attempts.

Task	Description
0. ingest_from_storage	Pull pharmacy files from Supabase Storage into a local working folder and a permanent dated archive
1. normalize_csvs	Standardize column names; merge and deduplicate pharmacy inventory files
2. dbt_seed	Load normalized CSVs into DuckDB
3. dbt_run	Build staging, core, and analytics models
4. dbt_snapshot	Capture price and availability history
5. push_to_postgres	Upsert core/analytics tables; append new history records

5. Pharmacy CSV Naming Convention

This is the contract the entire ingestion layer depends on. Files uploaded to Supabase Storage must follow:

```
pharmacy_inventory_pharmacy_<pharmacy_id>_<timestamp>.csv
```

Example: pharmacy_inventory_pharmacy_10_20260701.csv. A single fallback filename pharmacy_inventory.csv (no suffix) is also recognized. Structured, data-team-maintained files use fixed names with no variation: drugs.csv, drug_categories.csv, drug_alternatives.csv.

Pharmacy inventory files tolerate column name variation (e.g. "stock" is matched to quantity, "safety_stock" to minimum_stock) via fuzzy matching. Structured CSVs do not — their headers must match the schema exactly, since they are copied directly without normalization.

6. Key Design Assumption

Pharmacies are assumed to upload drug_id directly (an internal identifier), not a drug name — meaning pharmacies, or an upstream system, resolve drug names to the pipeline's internal ID before the file reaches storage. An inventory row whose drug_id does not exist in the drug catalogue is dropped rather than used to auto-create a new drug record, since a bare ID with no name provides nothing to catalogue. This assumption is flagged for confirmation with the backend team.

7. Technology Stack

Component	Technology
Transformation	dbt-duckdb 1.9.2 + DuckDB 1.2.1
Orchestration	Apache Airflow 3.1.1
Ingestion Storage	Supabase Storage
Production Warehouse	PostgreSQL (Neon)
Development Environment	Ubuntu Linux VM